Guild Audits

JANUARY
2026

FRACTAL LAUNCHPAD AUDIT REPORT

Prepared by: Joseph-leo, Theboiledcorn, 0xspryon, JustUzair, Adesola & Zainab
Idealz-codes

INTRODUCTION

A decentralized NFT launchpad platform enabling creators to deploy ERC721 and ERC1155 token collections with integrated a16z CantBeEvil licensing, upgradeability, and comprehensive security features.

DISCLAIMER

The Guild academy Fractal visions audit team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

AUDIT DETAILS

Audited the codebase at the commit hash

a6c4c379228b9e27fc26590e0f5f62c0eef971f8

SCOPE

This included the following files in scope

- src/a16z/CantBeEvilUpgradeable.sol
- src/Factory.sol
- src/FractalERC1155.sol
- src/FractalERC721.sol
- src/FractalLaunchpad.sol

SUMMARY OF FINDINGS

Severity breakdown

A total of **1** issue were identified and categorized based on severity

1 Informational

FINDINGS

INFO-1: UNAUTHORIZED CREATORS CAN CREATE ERC721 & ERC1155 TOKEN TYPE CONTRACTS WITHOUT PAYING ANY FEES.

SUMMARY

The launchpad protocol consists of the:

Factory: This is the contract responsible for launching the various token type contracts. Aka deploying minimal proxies that point to the reference ERC721 & ERC1155 contracts above. This contract is meant to be called by the launch pad. The method responsible is called createClone and is defined as follows:

```
function createClone(
 address _implementationContract,
 string memory _name, string memory _symbol, uint256 _maxSupply,
string memory _baseURI, address _owner, LicenseVersion _licenseVersion
 ) external returns (address) {
 if (_implementationContract == address(0)) revert
InvalidImplementation();
 if (_implementationContract.code.length == 0) revert
ImplementationHasNoCode();
 // convert the address to 20 bytes
 bytes20 implementationContractInBytes = bytes20(
 _implementationContract
 );
 //address to assign cloned proxy
 address proxy;
 // commented out for brevity
 return proxy;
 }
```

FractalLaunchpad: This is the contract responsible for collecting fees from unauthorized creators and delegating the actual token creation(proxy to the reference token contract creation) to the factory. The method that does this is called createLaunch and is defined as follows.

```
function createLaunch(
 string memory _name,
 string memory _symbol,
 uint256 _maxSupply,
 string memory _baseURI,
 LicenseVersion _licenseVersion,
 TokenType _tokenType
 ) external payable returns (uint256 launchId) {
 // if (_maxSupply == 0) revert MaxSupplyMustBeGreaterThanZero();
 launchId = nextLaunchId++;
// @audit: we collect fees from unauthorized users
 if(!authorizedCreators[msg.sender] && msg.sender != owner(){
 //charge fee
 if (msg.value < platformFee) revert InsufficientFee();
 (bool sent, ) = feeRecipient.call{value: platformFee}("");
 if (!sent) revert FailedToSendFee();
 }
 if(_tokenType == TokenType.ERC721) {
// @audit: we delegate the actual clone creation to the factory
contract.
 address tokenContract =
nftFactory.createClone(ERC721_IMPLEMENTATION, _name, _symbol,
_maxSupply,
_baseURI, msg.sender, _licenseVersion);
 // commented out for brevity
 } else {
// @audit: we delegate the actual clone creation to the factory
contract.
 address tokenContract =
nftFactory.createClone(ERC1155_IMPLEMENTATION, _name, _symbol,
_maxSupply,
_baseURI, msg.sender, _licenseVersion);
// commented out for brevity
 }
 }
```

Root cause:

The issue here is, the createClone method on the Factory contract has no access control to authorize calls only from the FractalLaunchpad. As such, a malicious user can just skip calling the FractalLaunchpad and directly call the createClone method with all the appropriate parameters and will end up with a clone of the appropriate token type without ever paying any fees.

Severity:

Information. Because the malicious user could just deploy a minimal proxy pointing to the reference token contracts directly without going through the factory as well.

Recommended mitigation:

Lockdown the createClone method to be callable only by the launchpad.